



PEC4: Introducción al aprendizaje computacional

Presentación

Cuarta PEC del curso de Inteligencia Artificial

Competencias

En esta PEC se trabajaran las siguientes competencias:

Competencias de grado:

- Capacidad de analizar un problema con el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y solucionarlo.

Competencias específicas:

- Conocer los diferentes aspectos de aprendizaje computacional (aprendizaje supervisado, no supervisado y por refuerzo).
- Conocer los fundamentos teóricos de los métodos más representativos.
- Conocer el tratamiento de los conjuntos de datos para la correcta validación de los sistemas de aprendizaje.

Objetivos

Esta PEC pretende evaluar diferentes aspectos de aprendizaje supervisado y no supervisado.

Descripción de la PEC a realizar

Arboles de Decisión

Pregunta 1) (3 puntos)

Un servidor de correo pretende crear un sistema para detectar correos SPAM que le llegan a sus usuarios a partir de ciertas características: máxima frecuencia de repetición de palabras es menor que 2 ($\text{freq} < 2$), la longitud de letras mayúsculas seguidas es mayor que 5 ($\text{long} > 5$), y la máxima frecuencia de repetición de palabras mayor que 10 ($\text{freq} > 10$). La siguiente tabla muestra un conjunto de emails que usaremos como conjunto de entrenamiento para detectar si el correo recibido es SPAM o no.



Clase	freq<2	long>5	freq>10
SPAM	Si	No	No
NO SPAM	No	No	No
SPAM	No	No	Si
SPAM	No	Si	Si
NO SPAM	No	No	No
NO SPAM	No	No	No
SPAM	Si	Si	No
NO SPAM	No	No	No
SPAM	No	No	No
SPAM	No	Si	No
SPAM	No	Si	No
SPAM	Si	Si	No
NO SPAM	No	No	No
NO SPAM	No	No	No
NO SPAM	No	No	No
NO SPAM	No	No	No

Construir un árbol de decisión a partir de este conjunto calculando la bondad de las particiones de los tres atributos binarios.

Solución:

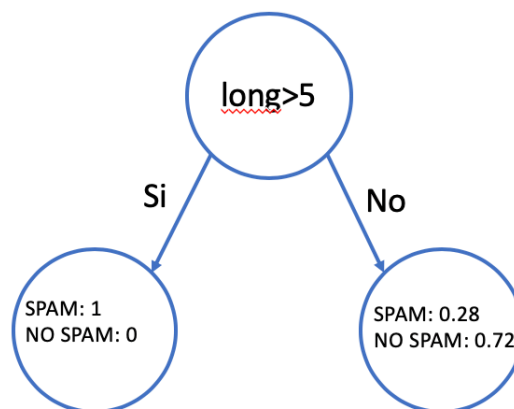
Raíz (Nivel 0)

En el **nodo raíz** calculamos la bondad de las particiones de los tres atributos para todos los elementos. El atributo freq<2 genera dos particiones SPAM Y NO SPAM donde el atributo No tiene 8 instancias con la clase NO SPAM y 5 instancias con SPAM y el atributo Si solo tiene 3 instancias con SPAM. La clase mayoritaria con el atributo No es NO SPAM y con el atributo Si es SPAM, y por lo tanto, la bondad de la partición es $(8+3)/16=0.68765$. Hacemos el mismo procedimiento para long>5 y freq>10. Para long>5, el atributo Si tiene 5 instancias con SPAM mientras que el atributo No tiene 8 NO SPAM y 3 SPAM. Cogiendo las clases mayoritarias de cada partición, nos queda $(8+5)/16=0,8125$. Finalmente, para freq>10, No tiene 8 instancias con



clase NO SPAM y 6 instancias con clase SPAM mientras que el atributo Si tiene 2 instancias de SPAM. Esto significa que la bondad de $\text{freq}>10$ es $(8+2)/16 = 0.625$.

Una vez hemos calculado la bondad para todos los atributos, seleccionamos el mejor atributo con la mejor bondad, $\text{long}>5$ con 0.8125 de bondad, y el árbol de decisión de nivel 0 es el siguiente:



Clase	<u>freq<2</u>	<u>long>5</u>	<u>freq>10</u>
SPAM	No	Si	Si
SPAM	Si	Si	No
SPAM	No	Si	No
SPAM	No	Si	No
SPAM	Si	Si	No

Clase	<u>freq<2</u>	<u>long>5</u>	<u>freq>10</u>
SPAM	Si	No	No
NO SPAM	No	No	No
SPAM	No	No	Si
NO SPAM	No	No	No
NO SPAM	No	No	No
NO SPAM	No	No	No
SPAM	No	No	No
NO SPAM	No	No	No
NO SPAM	No	No	No
NO SPAM	No	No	No

Donde la hoja de la izquierda clasifica correctamente todos los correos en SPAM, y por lo tanto terminal.

Nivel 1:

Como el nodo de la izquierda es terminal, seguimos con el mismo procedimiento con el nodo de la derecha, calculando la bondad para cada atributo con las muestras de este nodo.

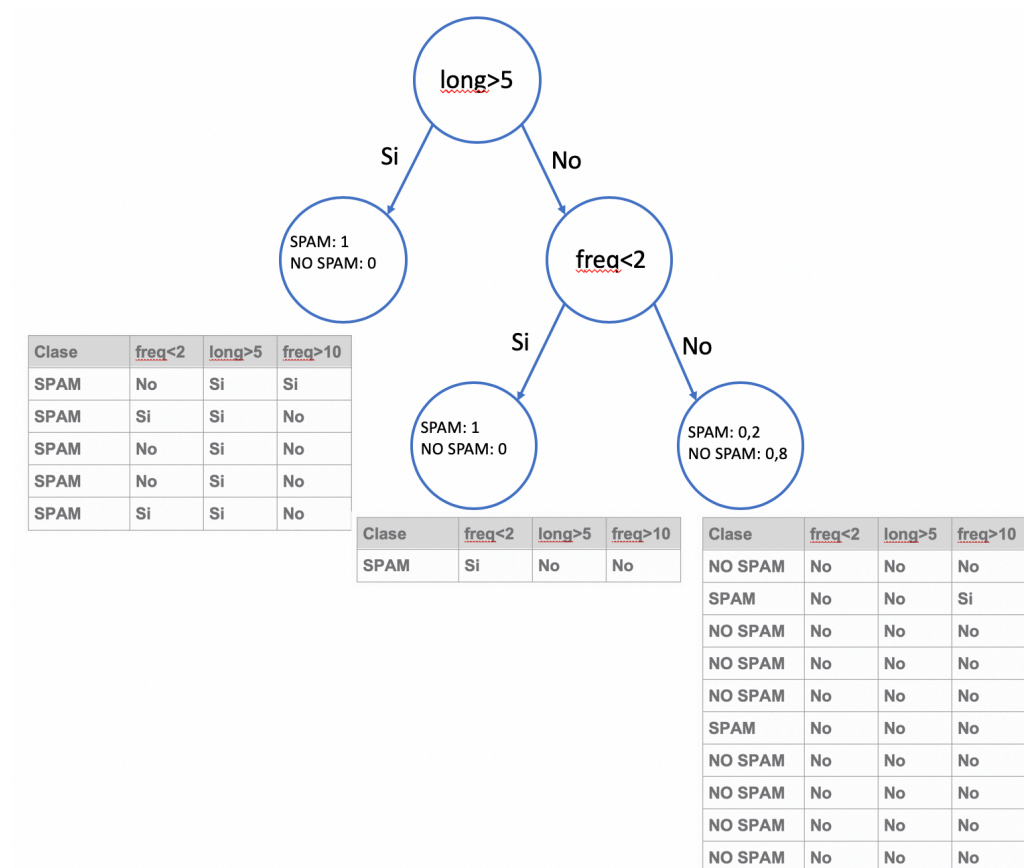


Bondad $\text{freq} < 2$: $(1+8)/11 = 0,81$

Bondad long 5: $(0+8)/11 = 0,72$

Bondad $\text{freq} > 10$: $(1+8)/11 = 0,81$

Hemos obtenido un empate entre dos de los atributos y podemos escoger cualquiera. Escogemos $\text{freq} < 2$ para hacer la división de las muestras y el árbol de decisión de nivel 1 es el siguiente:



Nivel 2:

Tal como ha sucedido en el nivel 1, el nodo del medio es terminal porque es puro completamente y seguimos calculando la bondad para los atributos del nodo de la derecha.

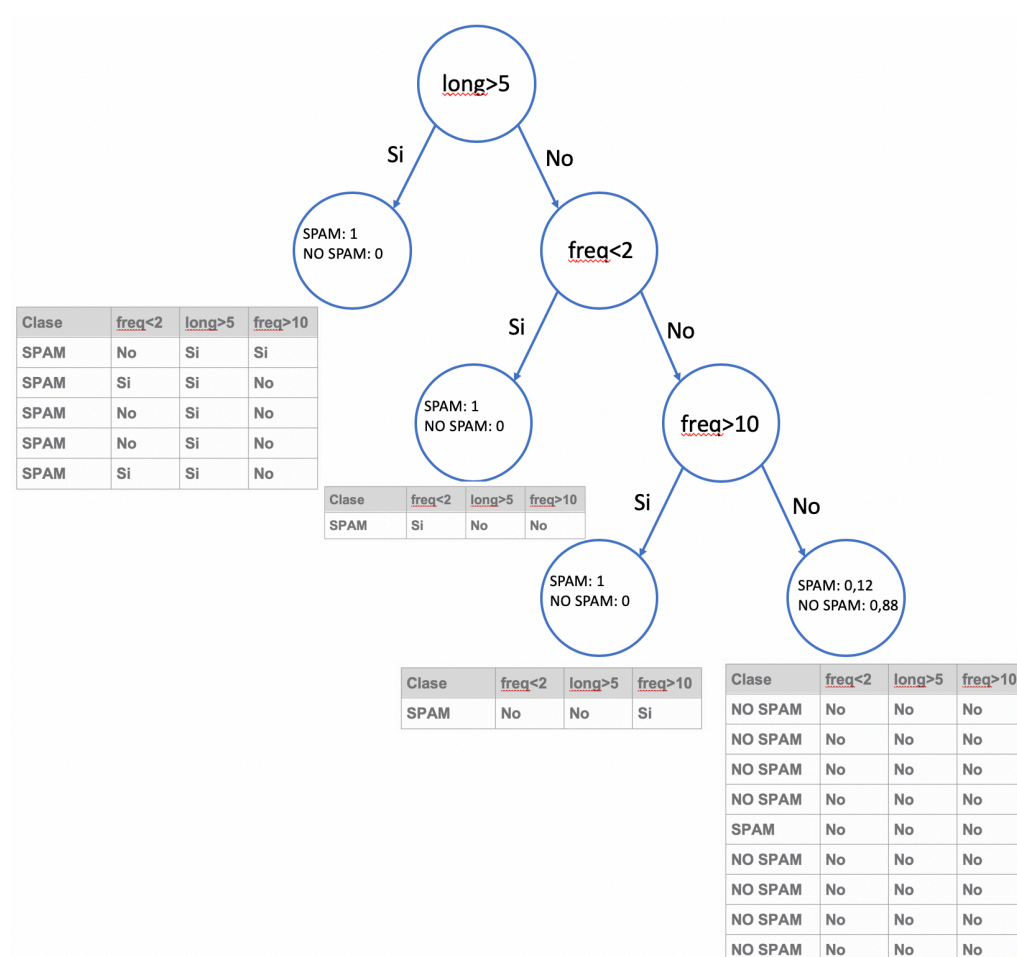
Bondad $\text{freq} < 2$: $(0+8)/10 = 0,8$

Bondad long 5: $(0+8)/10 = 0,8$

Bondad $\text{freq} > 10$: $(1+8)/10 = 0,9$



Por lo tanto, cogemos la $freq > 10$ para hacer la división del nodo y obtener la clasificación final de los correos entre SPAM y NO SPAM tal como muestra el siguiente árbol de decisión con las probabilidades de cada clase según el nodo terminal:



Pregunta 2) (2 punto)

En el ejercicio anterior se ha creado un árbol de decisión calculando la bondad de las particiones utilizando los atributos que han sido categorizados a variables binarias. En muchas aplicaciones estos datos son ordinarios (números enteros, reales, etc.). Por ejemplo, la frecuencia de una palabra en un correo, el numero medio de caracteres por palabra en el caso de detección de correos SPAM. Consideramos los siguientes dos atributos enteros: longitud de mayúsculas seguidas con rango $[1..100]$ y el numero máximo de repeticiones de una palabra con rango $[1..20]$, dos clases: SPAM y NO SPAM, como se calcularía la mejor partición de los datos para la clasificación de los



correos. (Nota: solo se usa el mejor atributo categorizado, código Python o pseudo-código es válido).

Solución:

Como hemos visto en la solución del anterior apartado es necesario calcular la bondad de una partición binaria, por ejemplo, $\text{freq} < 2$ o $\text{freq} > 2$ que hacen la misma partición de los datos, pero con valores inversos. Por lo tanto, consideramos solo el caso de frecuencia y longitud mayor ($>$) que cierto valor. Para calcular la bondad es necesario el vector de clase SPAM o NO SPAM (y_{clase}) y un vector binarizado que determina la partición de los datos (x_{atributo}), por ejemplo, $\text{long} > 5$. Dicha función se puede calcular de la siguiente manera:

```
def bondad(x_atributo, y_clase):
    n_instancias=len(x_atributo)
    x_0 = np.zeros(2,) # particion igual a 0, con dos clases: 0 o 1
    x_1 = np.zeros(2,) # particion igual a 1, con dos clases: 0 o 1
    for i_idx in range(n_instancias):
        if x_atributo[i_idx]==0:
            y_i = y_clase[i_idx]
            x_0[y_i]+=1
        else:
            y_i = y_clase[i_idx]
            x_1[y_i]+=1
    bondad = (np.max(x_0)+np.max(x_1))/n_instancias
```

Como se ha calculado anteriormente la bondad en la pregunta 1, la implementación de la bondad es opcional, aunque se tiene que llamar y pasar los parámetros ($x_{\text{atributos}}$ y y_{clase}).

Una vez tenemos definida la función bondad, se ha de binarizar cada atributo para cada valor de su rango. Es decir, la binarización consiste en que todo elemento mayor o igual a un valor es igual a 1, y los menores a 0. El atributo longitud (x_{longitud}) se ha de binarizar para cada valor entre 1 y 100 mientras que el atributo frecuencia ($x_{\text{frecuencia}}$) entre 1 y 20. La binarización se puede programar como:

```
def binarizacion(x, thr)
    out = np.zeros(x.shape)
    out[x>=thr]=1
    return out
```

Para calcular la mejor partición de los datos pero con atributos enteros, hemos de calcular la bondad para cada partición obtenida aplicando la binarización del atributo en el rango del atributo, y escoger el máximo entre los dos atributos. Dicha operación se calcula de la siguiente manera:



```
# y_clase, x_longitud, x_frecuencia se suponen que estan en memoria o pasada por parametros
bondad_longitud = np.zeros(100,)
bondad_frecuencia = np.zeros(20,)

#Se calcula la bondad para todo el rango de longitud
for i_longitud in range(100):
    x_atributo = binarizacion(x_longitud, i_longitud)
    bondad_longitud[i_longitud] = bondad(x_atributo, y_clase)

#Se calcula la bondad para todo el rango de frecuencia
for i_frecuencia in range(20):
    x_atributo = binarizacion(x_frecuencia, i_frecuencia)
    bondad_frecuencia[i_frecuencia] = bondad(x_atributo, y_clase)

idx_max_longitud_bondad = np.argmax(bondad_longitud)
valor_max_longitud = bondad_longitud[idx_max_longitud_bondad]

idx_max_frecuencia_bondad = np.argmax(bondad_frecuencia)
valor_max_frecuencia = bondad_frecuencia[idx_max_frecuencia_bondad]

print('la particion de los datos se calcula con:')
if valor_max_frecuencia > valor_max_longitud:
    print('frecuencia mayor o igual a '+str(idx_max_frecuencia_bondad))
else:
    print('longitud mayor o igual a '+str(idx_max_longitud_bondad))
```

Pregunta 3) (3 puntos)

Utilizando el notebook de Python que evalúa el k nearest neighbor, implementar aquello que haga falta para la implementación del k-means. (Nota: considerar los clusters como el conjunto de datos del kNNs y todos los puntos del conjunto de datos como queries).

Solución:

Primero de todo se ha de asignar el cluster mas cercano a cada instancia de la base de datos. Esto se puede hacer usando el kNN donde el query es la instancia actual y la base de datos son todos los centroides, tal como se puede ver abajo.

```
idx2cluster = np.zeros((n_instances,))
for i_instances in range(n_instances):
    idx2cluster[i_instances] = knearestneighbors(data[i_instances,:], clusters, n_neighbors=1)
```

Una vez tenemos todas las instancias asignadas a los clusters, calculamos el *i-th* cluster como la media de todos puntos asignados al cluster *i* y así para todos los clusters.

```
for i_clusters in range(n_clusters):
    clusters[i_clusters,:] = np.mean(data[idx2cluster==i_clusters,:],axis=0)
```

Toda la solución esta en el ipython notebook.

Pregunta 4) (2 puntos)

Utilizando el programa Python adjunto, que evalúa el kNNs, Arboles de Decision y redes neuronales con una evaluación simple. Implementar aquello que haga falta para poder evaluar estos algoritmos utilizando una validación cruzada (cross validation o K-fold cross validation), con una implementación



genérica para K y evaluada con K=10. Además, calcular la media y la variancia de la precisión de cada uno de los splits y de cada clasificador. (Nota: se recomienda usar la clase KFold de sklearn por su simplicidad)

Solución:

Primero de todo se ha de definir la variable con el numero de splits K=10, y generar los splits, y las variables para almacenar la precisión de cada clasificador.

```
K=10
kf = KFold(n_splits=K) # Define the split - into 2 folds
# important point to generate the splits
#kf.get_n_splits(X)
knn_score = np.zeros((K,))
tree_score = np.zeros((K,))
NN_score = np.zeros((K,))
```

Una vez tenemos los splits, iteramos por los K splits para entrenar los clasificadores y calcular la precisión en el split de test para calcular la media y la variación estándar al final.

```
# K-fold Crossvalidation
from sklearn.model_selection import KFold
K=10
kf = KFold(n_splits=K) # Define the split - into 2 folds
# important point to generate the splits
#kf.get_n_splits(X)
knn_score = np.zeros((K,))
tree_score = np.zeros((K,))
NN_score = np.zeros((K,))
for i_iter, (train_index, test_index) in enumerate(kf.split(X)):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
    clf_knn.fit(X_train, y_train)
    clf_tree.fit(X_train, y_train)
    clf_NN.fit(X_train, y_train)
    y_pred = clf_knn.predict(X_test)
    print("Fold "+str(i_iter))
    score=accuracy_score(y_test, y_pred)
    knn_score[i_iter]=score
    print("KNN Accuracy:"+str(score))
    y_pred = clf_tree.predict(X_test)
    score=accuracy_score(y_test, y_pred)
    tree_score[i_iter]=score
    print("Decision Tree Accuracy:"+str(score))
    y_pred = clf_NN.predict(X_test)
    score=accuracy_score(y_test, y_pred)
    NN_score[i_iter]=score
    print("NN Accuracy:"+str(score))
print('KNN classifiers accuracy: '+str(np.mean(knn_score))+' +/- '+str(np.std(knn_score)))
print('Tree classifier accuracy '+str(np.mean(tree_score))+' +/- '+str(np.std(tree_score)))
print('NN classifier accuracy: '+str(np.mean(NN_score))+' +/- '+str(np.std(NN_score)))
```

Toda la solución esta en el ipython notebook.

Recursos

Para hacer esta PEC el material imprescindible es el módulo 5 y para ejecutar los notebooks en Python del ejercicio recomendó usar Google Colab que es gratuito (<https://colab.research.google.com>).



Criterios de valoración

Las puntuaciones se muestran en cada pregunta del enunciado.

Formato y fecha de entrega

Para dudas y aclaraciones sobre el enunciado, dirigiros al consultor responsable del aula.

Hay que entregar la solución en un archivo PDF usando una de las plantillas entregadas conjuntamente con este enunciado. Adjuntar el fichero a un mensaje en el apartado Entrega y Registro de EC (REC).

El nombre del archivo debe ser *Apellidos_Nombre_IA_PEC4* con la extensión .pdf (PDF).

La fecha límite de entrega es el: **22/12/2019** (a las 24 horas).

Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.

Nota: **Propiedad intelectual**

A menudo es inevitable, al producir una obra multimedia, hacer uso de recursos creados por terceras personas. Es por tanto comprensible hacerlo en el marco de una práctica de los estudios de Informática, siempre que se documente claramente y no suponga plagio en la práctica.

Por lo tanto, al presentar una práctica que haga uso de recursos ajenos, se presentará junto con ella un documento en el que se detallen todos ellos, especificando el nombre de cada recurso, su autor, el lugar donde se obtuvo y el su estatus legal: si la obra está protegida por copyright o se acoge a alguna otra licencia de uso (Creative Commons, licencia GNU, GPL ...).

El estudiante deberá asegurarse de que la licencia que sea no impide específicamente su uso en el marco de la práctica. En caso de no encontrar la información correspondiente deberá asumir que la obra está protegida por copyright.

Deberán, además, adjuntar los archivos originales cuando las obras utilizadas sean digitales, y su código fuente esté corresponde.